# Git-Grundlagen

Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

## Ziel

Du kannst Änderungen im eigenen Lern-Repository über die GitHub-Oberfläche nachvollziehen, auf einem Arbeitsbranch speichern und als Pull Request zur Übernahme vorschlagen.

## Was macht Git?

Git ist ein Versionsverwaltungssystem. Es speichert Änderungen an Dateien als **Commits** (gesicherte Projektstände), sodass du den Verlauf ansehen und Änderungen vergleichen kannst. GitHub ist ein Webdienst, auf dem ein Git-Repository gespeichert und gemeinsam bearbeitet werden kann.

In diesem Kurs arbeitest du in deinem eigenen Lern-Repository. Das Kurs-Repository enthält die Vorlagen. Für dein Lern-Repository brauchst du keinen Git Flow.

## Repository und Dateien auf GitHub

Ein **Repository** (kurz: Repo) enthält Projektdateien und deren Git-Verlauf. In der GitHub-Weboberfläche kannst du Dateien anzeigen, neue Dateien anlegen und vorhandene Dateien bearbeiten. Die Kursvorbereitung in Teil 1 zeigt, wie du dein Lern-Repository einrichtest und Dateien im Browser speicherst.

**Aufgabe:** Öffne dein Lern-Repository auf GitHub und finde den Standardbranch sowie die vorhandene `README.md`.

**Ausführung:** Öffne den Tab **Code**. Nutze die Branch-Auswahl, um den aktuellen Branch zu sehen, und öffne anschließend `README.md`.

**Kontrolle:** Du siehst die Dateien des ausgewählten Branches. Der Standardbranch ist der Branch, den GitHub als Hauptversion des Repositorys verwendet; häufig heißt er `main`.

### Änderungen und Commits

Ein **Commit** ist ein gespeicherter Stand der Änderungen. Auf GitHub kannst du eine Datei bearbeiten und mit **Commit changes** speichern. Die Weboberfläche erstellt den Commit direkt; der lokale Git-Zwischenschritt **Staging** wird dabei nicht separat angezeigt.

Ein Commit gehört zu einem Branch. Die Commit-Nachricht sollte knapp beschreiben, was geändert wurde, zum Beispiel `Ergänze YAML-Schrittbeschreibung`.

### Remote, Push und Pull

Ein **Remote** ist ein anderes Git-Repository, das als Ziel für den Austausch von Änderungen eingetragen ist. Im Kurs ist das dein online gespeichertes Lern-Repository auf GitHub. **Push** überträgt lokale Commits zum Remote; **Pull** holt Änderungen vom Remote in ein lokales Repository. Diese Begriffe gehören zum lokalen Git-Ablauf. Wenn du im Browser auf GitHub bearbeitest und commitest, speicherst du direkt in dem online ausgewählten Repository und Branch – ein separater Push-Schritt ist dort nicht sichtbar.

Ein Pull Request ist etwas anderes als `git pull`: Ein Pull Request schlägt Änderungen aus einem Branch zur Prüfung und möglichen Übernahme in einen anderen Branch vor.

Als Anwendung danach erstellst du die `README.md` in Markdown-Grundlagen. Verwende dort die GitHub-Oberfläche, um die README im eigenen Lern-Repository zu speichern. Die Commit-Schritte und die Kontrolle im Verlauf sind in dieser Lerneinheit beschrieben.

### Branches: getrennte Arbeitslinien

Ein **Branch** ist eine benannte Entwicklungslinie im Repository. Auf einem Arbeitsbranch kannst du Änderungen vorbereiten, ohne sie sofort in den Standardbranch zu übernehmen. Die Dateien und der Verlauf können sich je nach ausgewähltem Branch unterscheiden.

**Aufgabe:** Erstelle in deinem Lern-Repository einen Arbeitsbranch für eine Kursänderung.

**Ausführung:** Öffne die Branch-Auswahl auf dem Tab **Code**, gib einen neuen Namen ein und wähle die angezeigte Möglichkeit, den Branch vom Standardbranch aus zu erstellen. Die genaue Beschriftung der Oberfläche kann sich ändern. Wechsle anschließend in den neuen Branch.

**Kontrolle:** Die Branch-Auswahl zeigt den neuen Namen als aktuell ausgewählten Branch. Prüfe vor dem Bearbeiten, dass du im richtigen Repository und Branch bist.

Lege neue Branches im Lern-Repository nach den Anweisungen der jeweiligen Kursaufgabe an. Die Git-Flow-Regeln zur Pflege dieses Kurs-Repositories gelten nicht für dein Lern-Repository. In diesem Grundlagenmodul ist der Ablauf auf GitHub zentral; die lokalen Begriffe zu Remote, Push und Pull ordnen einen möglichen späteren Terminalweg ein.

## Pull Request: Änderung zur Prüfung vorschlagen

Ein **Pull Request** (PR) ist ein Vorschlag, Änderungen aus einem Branch in einen anderen Branch zu übernehmen. Auf GitHub können Beteiligte die Änderungen ansehen, besprechen und prüfen, bevor sie zusammengeführt werden.

**Aufgabe:** Erstelle einen Pull Request für eine kleine Änderung in deinem Übungsbranch.

**Ausführung:** Ändere eine Datei im Arbeitsbranch und speichere die Änderung mit **Commit changes**. Öffne danach den Bereich **Pull requests** und erstelle einen Vorschlag, der deinen Arbeitsbranch als Quellbranch und den Standardbranch als Zielbranch verwendet. Lies vor dem Erstellen die angezeigten Branch-Namen und die Dateiansicht aufmerksam.

**Kontrolle:** Der Pull Request zeigt den Titel, die Branches und die geänderten Dateien. Auf der Registerkarte **Files changed** kannst du prüfen, ob nur die beabsichtigte Änderung enthalten ist. Einen Pull Request zu erstellen führt nicht automatisch dazu, dass die Änderung in den Standardbranch übernommen wird; das geschieht erst durch das Zusammenführen (Merge).

Erstelle in deinem eigenen Lern-Repository keinen Pull Request, wenn die Kursaufgabe stattdessen ausdrücklich eine direkte Änderung oder einen anderen Ablauf verlangt.

## Der Git-Ablauf auf einen Blick

1. **Branch wählen:** Auf dem vorgesehenen Branch arbeiten.
2. **Datei ändern:** Inhalt im Browser oder lokal bearbeiten.
3. **Commit erstellen:** Änderung mit einer Nachricht speichern. Auf GitHub geschieht das über **Commit changes**.
4. **Änderung vorschlagen:** Bei einem Pull Request den Unterschied prüfen und besprechen.
5. **Zusammenführen:** Nur wenn es die Aufgabe vorsieht, den Pull Request in den Zielbranch integrieren.

GitHub-Oberfläche und lokale Git-Befehle sind verschiedene Arbeitsweisen. Lokal gibt es unter anderem einen Staging-Bereich, in dem ausgewählt wird, welche Änderungen in den nächsten

Commit kommen. Der Browserablauf zeigt diesen Schritt nicht separat. Diese Anleitung behandelt den Browserweg; ein Terminalablauf wäre eine eigene ergänzende Anleitung.

### Workflow-Dateien und GitHub Actions

Eine Workflow-Datei ist eine Datei im Repository. Ein Commit mit einer Workflow-Änderung ändert deshalb den gespeicherten Inhalt des Branches. GitHub Actions verarbeitet die Workflow-Datei abhängig von ihren Triggern.

Für einen manuellen Start mit `workflow_dispatch` muss die Workflow-Datei auf dem Standardbranch liegen. Danach kann beim manuellen Start ein Branch ausgewählt werden. Ein Commit auf einem Arbeitsbranch allein macht den Workflow daher nicht automatisch im Actions-Tab manuell startbar. Weitere Einzelheiten stehen in der Vorbereitung von Teil 1.

In Teil 2, Lektion 8, sind `push`, `pull_request` und `schedule` ausdrücklich als Ausnahme von der Kursregel dokumentiert. Die folgende Übung ergänzt keinen Trigger; sie verwendet nur den bereits vorhandenen manuellen Start.

#### Brückenübung: Workflow-Änderung mit Git sichern und prüfen

**Lernziel:** Du änderst einen ungefährlichen YAML-Wert auf einem Arbeitsbranch, prüfst den Datei-Diff und kontrollierst die Änderung anschließend im manuellen Workflow-Lauf.

**Voraussetzung:** Die Datei `.github/workflows/01-hallo-linux.yml` wurde bereits auf dem Standardbranch deines Lern-Repositorys angelegt und committet. Der Workflow enthält `workflow_dispatch`. Erstelle für diese Übung einen neuen Arbeitsbranch vom aktuellen Standardbranch, damit der Branch die Workflow-Datei enthält. Prüfe in der Dateiansicht des neuen Branches, dass die Datei vorhanden ist, bevor du sie bearbeitest. Auf GitHub muss die Datei außerdem auf dem Standardbranch liegen, damit der manuelle Start angeboten wird. Im Dialog **Run workflow** kannst du über das Feld **Branch** den Arbeitsbranch auswählen. GitHub führt dabei den Workflow aus der gewählten Referenz aus. Siehe GitHubs aktuelle Anleitung zum manuellen Starten eines Workflows[1] und zum Ansehen von Workflow-Läufen[2] (Prüfstand: 23.10.2026).

**Datei ändern:** `.github/workflows/01-hallo-linux.yml` im eigenen Lern-Repository.

[1] https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow
[2] https://docs.github.com/en/actions/how-tos/monitor-workflows/viewing-workflow-run-history

**Aufgabe:** Ändere auf einem Arbeitsbranch den Namen eines vorhandenen Schritts, zum Beispiel `Begrüßung` in `Begrüßung aus meinem Arbeitsbranch`. Der Wert `name` ändert nur die Beschriftung des Schritts, nicht den ausgeführten Befehl.

**Ausführung:**

1. Erstelle über die GitHub-Oberfläche einen Arbeitsbranch vom Standardbranch und wechsle zu ihm.
2. Öffne `.github/workflows/01-hallo-linux.yml` und bearbeite den Wert bei `name:` des Schritts `Begrüßung`. Belasse die Einrückung unverändert. Für den Beispielnamen sind keine Anführungszeichen erforderlich; wenn du Sonderzeichen wie `:` verwendest, setze den Wert in Anführungszeichen.
3. Speichere die Änderung mit **Commit changes** auf dem Arbeitsbranch. Verwende eine kurze Commit-Nachricht, zum Beispiel `Benenne Begrüßungsschritt um`.
4. Öffne den gespeicherten Commit und prüfe in der Diff-Ansicht zuerst, dass nur der beabsichtigte `name:`-Wert geändert wurde.
5. Öffne **Actions**, wähle **01 - Hallo Linux** und klicke **Run workflow**. Wähle unter **Branch** deinen Arbeitsbranch und starte den Lauf.
6. Öffne den Lauf und den Job `erkunden`. Prüfe in der Schrittliste, dass der neue Name erscheint. Der Schritt führt weiterhin `echo "Hallo aus dem Linux-Runner!"` aus.

**Ergebniskontrolle – Stufe 1, Diff:** Im Commit-Diff ist nur der Anzeigename des Begrüßungsschritts geändert. Es gibt keine Änderungen an Trigger, Job oder `run`-Befehl.

**Ergebniskontrolle – Stufe 2, Workflow-Lauf:** Der manuelle Lauf ist erfolgreich und der Job zeigt den geänderten Schritt-Namen. Damit hast du sowohl die gespeicherte Änderung als auch deren Verwendung im Lauf geprüft.

**Typische Fehler:**

- **Schrittname fehlt oder YAML kann nicht verarbeitet werden:** Vergleiche die Einrückung mit dem ursprünglichen Schritt und prüfe, dass `name:` samt Doppelpunkt erhalten blieb.
- **Name enthält einen Doppelpunkt und wird falsch gelesen:** Setze den vollständigen Textwert in Anführungszeichen.
- **Der Lauf zeigt den alten Namen:** Prüfe, ob der Commit wirklich auf dem Arbeitsbranch gespeichert wurde und ob du im Dialog **Run workflow** denselben Branch ausgewählt hast.
- **Die Änderung taucht nicht im Commit-Diff auf:** Wahrscheinlich wurde die Bearbeitung nicht mit **Commit changes** gespeichert. Kehre zum Editor zurück und speichere sie auf dem Arbeitsbranch.

- **Die Workflow-Datei fehlt im Arbeitsbranch:** Prüfe, ob der Branch neu vom aktuellen Standardbranch erstellt wurde. Erstelle andernfalls einen neuen Branch vom Standardbranch und kontrolliere erneut, dass die Datei sichtbar ist.
- **Run workflow wird nicht angeboten:** Prüfe, ob die Workflow-Datei mit `workflow_dispatch` auf dem Standardbranch liegt.

Du kannst danach optional einen Pull Request öffnen, um die Änderung zur Übernahme in den Standardbranch vorzuschlagen. Führe das Zusammenführen nur aus, wenn es die Kursaufgabe verlangt.

### Änderungen kontrollieren und sicher arbeiten

Bevor du einen Commit oder Pull Request speicherst, prüfe:

- Ist das richtige Repository geöffnet?
- Ist der richtige Branch ausgewählt?
- Zeigt die Dateiansicht nur die beabsichtigten Änderungen?
- Enthält die Commit-Nachricht eine klare Beschreibung?

Übe zunächst an einer kleinen Änderung im eigenen Lern-Repository. Verwende keine Befehle oder Schaltflächen, deren Wirkung du nicht kennst, um Änderungen zu verwerfen oder den Verlauf umzuschreiben. Insbesondere können Befehle wie `git reset --hard` ungesicherte Arbeit löschen; `git push --force` kann einen veröffentlichten Verlauf verändern.

### Weiterlernen

Die Oberfläche von GitHub ändert sich gelegentlich. Für aktuelle Schritte siehe die offizielle GitHub-Dokumentation zu Branches[3] und Pull Requests[4]. Für lokale Git-Grundlagen bietet *Pro Git* die Abschnitte zu Git-Grundlagen[5] und Branches[6].

[3] https://docs.github.com/en/pull-requests/how-tos/commit-changes/managing-branches-within-your-repository
[4] https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests
[5] https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository
[6] https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell